Schilmoeller & Schoenfield, PC

Staff Portal & AI Communications Agent

Handoff & Migration Guide

Prepared for: The receiving engineer and the Schilmoeller & Schoenfield team

Prepared by: PRIME (RJ Tohay — rj@primelive.ai; early phases by Nate Geraldez — nate@primelive.ai)

Date: 2026-06-04 (rev. 4)

Goal: Migrate the system from PRIME's Railway environment to the firm's own server, and hand over day-to-day operation.

> *Companion documents:* ***README.md*** *(technical deep-dive) and* ***USER_GUIDE.pdf / docs/USER_GUIDE.md*** *(end-user app guide + integrations setup). This document covers what those don't: access, migration steps, operations, and the sharp edges.*

1. What this system is

An AI-assisted email platform for the firm's Microsoft 365 mailbox (`jane@schilcpa.com`). It polls the mailbox, categorizes every incoming email with AI (Anthropic Claude), assigns a triage tier, drafts a suggested reply from the firm's knowledge base, and gives staff a review UI. **Nothing is ever sent without human approval** (independent safety gates — see §6.2). It also supports composing new outbound email, attachments in both directions, spam/delete synced back to Outlook, per-client saved folders, full-text search, an escalation queue for Jane, **per-user email signatures** (the sender's signature is appended at send; auto-sent mail uses the company signature), an **Analytics page** (AI token usage vs. budget, email volume, draft quality, escalation trends), and a complete audit trail.

- **Backend:** Python 3.12 / FastAPI / SQLAlchemy / Alembic — `backend/`
- **Frontend:** Next.js 14 / TypeScript / Tailwind — `frontend/`
- **Database:** PostgreSQL 16 (13 tables, migrations 001–018)
- **In-app user guide:** the app ships with an interactive **/tutorials** page — that, plus the User Guide, is the end-user manual for Jane and staff.

2. Repositories & git workflow

Remote	URL	Role
`origin` (fan-out)	`github.com/rheynardjan/schillerCPA` **and** `github.com/Janedev99/Communications-Automation`	One `git push` updates both
Client's copy	`github.com/Janedev99/Communications-Automation`	**The authoritative copy the client owns. Migrate from here.**

- **Branches:** `master` = production, `development` = integration. Feature work happens on `FEAT/` , `FIX/` , `CHORE/` , `DOCS/` branches off `development` ; merges are `--no-ff` .
- `master` is only updated from `development` with the firm's approval.
- After handoff, the receiving engineer can simplify to whatever workflow they prefer — the only hard rule worth keeping: **don't push straight to `master` without testing**, because production migrations run automatically on deploy (§4).

3. Access checklist — what the receiving engineer needs

#	Item	From	Notes
1	GitHub access to `Janedev99/Communications-Automation`	Jane / Craig	Already in the client's org
2	Railway project access (or a fresh DB dump at cutover)	PRIME (RJ)	Needed once, for the data migration
3	**Azure app registration credentials** (`MSGRAPH_TENANT_ID`, `MSGRAPH_CLIENT_ID`, `MSGRAPH_CLIENT_SECRET`)	PRIME (RJ) → rotate after	App permissions: `Mail.Read`, `Mail.Send`, `Mail.ReadWrite` (application, admin-consented). **Rotate the client secret after migration** so PRIME no longer holds a working credential. The registration lives in the firm's Azure tenant — it keeps working from any host.

4	**Anthropic API key**	Firm should create its own	console.anthropic.com → API Keys. Don't inherit PRIME's key — billing and rate limits should be the firm's.
5	`APP_SECRET_KEY`	Generate fresh	`python -c "import secrets; print(secrets.token_hex(32))"` — do NOT reuse PRIME's
6	RunPod account	Already held client-side	Optional — only if the self-hosted LLM path is ever revived (§6.5). The pod used during development is on a shared account the firm's engineering contact already holds.
7	Slack webhook URL (escalation notifications)	Firm's Slack admin	Optional — `SLACK_WEBHOOK_URL`
8	Admin login for the app	Set via `ADMIN_EMAIL`/`ADMIN_PASSWORD` env + `seed_admin.py`	Existing user accounts come across with the DB dump

4. Current production environment (what you're migrating FROM)

Hosted on **Railway**, three services:

Service	Build	Start
Backend	`backend/Dockerfile`	`sh -c 'alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000} --workers 1'`
Frontend	`frontend/Dockerfile` (multi-stage, Next standalone)	`node server.js`
Postgres	Railway plugin	`DATABASE_URL` injected

Three things to internalize about this setup:

1. **Migrations run on every backend deploy** (`alembic upgrade head` in the start command). Convenient, but a bad migration can block boot — the health check (`/health` , 300s timeout) will catch it.
2. **`NEXT_PUBLIC_API_URL` is a Docker BUILD ARG**, not a runtime env var. It's inlined into the JS bundle at build time. On Railway it's set under *Build* settings; on any new host it must be passed to `docker build` . Getting this wrong produces a frontend that builds fine but talks to the wrong backend.
3. **Deploy drift has happened before.** In May 2026 production silently ran a stale build while master had moved on (migration lag was the symptom). After every deploy, verify the running version actually changed — check a recently-shipped feature or the migration level (`SELECT version_num FROM alembic_version;` should read `018` as of this writing). `scripts/post_deploy_verify.py` automates part of this (§9).

5. Migration runbook (Railway → client server)

5.1 Recommended target: docker-compose

The repo ships a production-shaped `docker-compose.yml` : Postgres 16 + backend + frontend + **Caddy** (reverse proxy with automatic TLS on ports 80/443). This is the lowest-friction self-hosting path.

Alternative: run the two Dockerfiles under any orchestrator, or bare-metal with systemd — the only invariants are PostgreSQL 16+, a single backend worker, and the env vars.

5.2 Step-by-step

Prepare (no downtime):

1. Provision the host (Docker + docker-compose). Open 80/443.
2. Clone `Janedev99/Communications-Automation` .
3. `cp .env.example .env` and fill in everything (§3 items), then run `python scripts/preflight_check.py` to validate the values (§9). Production checklist:
   - `APP_ENV=production`
   - `APP_SECRET_KEY` = freshly generated (boot **fails** on the dev default in production)
   - `LLM_PROVIDER=anthropic` , `ANTHROPIC_API_KEY` = the firm's key
   - `EMAIL_PROVIDER=msgraph` + the four `MSGRAPH_*` values
   - `SHADOW_MODE=true` (keep auto-send off — see §6.2)
   - `CORS_ORIGINS` = the real frontend URL only (no localhost)
   - `NEXT_PUBLIC_API_URL` = the real backend URL (build arg!)

- `TRUSTED_PROXIES` = Caddy's container IP for accurate client IPs (blank is safe)

4. Build images. Do a dry-run boot against an EMPTY local database first. **Tables are created automatically** — the backend's start command runs `alembic upgrade head` before the server boots, which applies migrations 001→018 in order and creates the full schema on an empty database. No manual `CREATE TABLE` work is ever needed. **But verify, don't trust.** After the dry-run boot, double-check the schema actually materialized: `SELECT version_num FROM alembic_version;` — must read **018** `SELECT tablename FROM pg_tables WHERE schemaname = 'public' ORDER BY 1;` Expect exactly these **13 application tables** (plus `alembic_version`): `ai_budget_usage`, `audit_log`, `draft_responses`, `email_messages`, `email_threads`, `escalations`, `knowledge_entries`, `runpod_daily_usage`, `runpod_state`, `sessions`, `system_settings`, `tier_rules`, `users`. If any are missing or `alembic_version` reads lower than 018, the migration step failed silently — check the backend container logs for the alembic output before going any further. Then confirm `/health` passes.

Cutover (brief downtime, do it outside business hours):

5. **Stop the Railway backend service first.** The poller marks emails processed as it ingests them — two pollers against two databases would each process new mail independently. Stop it, then dump.
6. Dump production data: `pg_dump "$RAILWAY_DATABASE_URL" --no-owner --no-privileges -Fc -f jane_prod.dump`
7. Restore into the new Postgres: `pg_restore --no-owner --no-privileges -d "$NEW_DATABASE_URL" jane_prod.dump`
8. Start the new stack. The backend's `alembic upgrade head` will no-op (the dump already contains schema + `alembic_version`). **Re-run the table-verification SQL from step 4** against the restored database — same 13 tables, `alembic_version` = `018`, and row counts that look like production (e.g. `SELECT count(*) FROM email_threads;` should match what the old dashboard showed).
9. **Verify** (15 minutes, with Jane or Sara available) — run `scripts/post_deploy_verify.py` (§9), then manually:
   - `GET /health` returns ok; login works with an existing account
   - Dashboard shows the historical threads/stats (proves the data came across)
   - Send a test email TO the mailbox from a personal account → it appears in the inbox within ~60s with a category and a draft (proves poller + AI)
   - Open an old message with an attachment → download works (proves Graph credentials)
   - Reply-send the test thread WITH an attachment (Jane approves) → recipient receives it; sent bubble shows the attachment chip (proves the send path)
   - Check Settings → Integrations: all cards green
10. Update Jane's browser bookmark (and staff bookmarks) to the new URL. The app is accessed by bookmark — there is no other entry point to update.
11. Decommission: delete the Railway services, **rotate the Azure client secret**, and PRIME removes its copies of all credentials.

5.3 Rollback

Until step 11, the Railway environment is intact (just stopped). Rollback = stop new stack, restart Railway backend, revert bookmarks. The only divergence risk is mail that arrived during cutover — the poller backfills on restart since it dedupes by Message-ID, so nothing is lost either way.

6. Operations guide

6.1 Background jobs (all in-process — hence single worker)

Job	Interval	What it does
Email poller	60s (`EMAIL_POLL_INTERVAL_SECONDS`)	Fetch → dedupe → categorize → tier → draft → escalate
Session cleanup	hourly	Purges expired login sessions
RunPod watchdog	60s	Idle-stops the GPU pod (no-op when `RUNPOD_POD_ID` empty)

Never run more than one backend worker/replica — each would run its own poller.

6.2 The auto-send gates (IMPORTANT)

Auto-sending of T1 drafts only happens if **all** of these are open:

1. `SHADOW_MODE=false` (environment variable)
2. `auto_send_enabled=true` (Settings UI → system_settings table)
3. The email's category has `t1_eligible=true` in Triage Rules **and** the AI's confidence beat that category's threshold. Every category defaults to OFF, and `complaint` / `urgent` / `promotional` are hard-locked at the API layer — they can never be enabled.

Production today: `SHADOW_MODE=true` → nothing auto-sends, every email is human-approved. **Do not change without Jane's explicit sign-off.** Drafts still generate in shadow mode — that's by design.

6.3 AI budget

`DAILY_TOKEN_BUDGET` (default 1,000,000 tokens/day, resets UTC midnight) caps all AI spend. Exhausted → categorization falls back to the keyword rules engine; drafting pauses until reset. The **Analytics page** charts usage against the budget. Bulk reprocessing scripts can burn the full budget in one run — plan those.

6.4 Routine admin tasks

- **Signatures (per-user since migration 018)**: every user edits their own under Settings → "My Signature"; admins edit the firm-level **company signature** (used for T1 auto-sent mail and as the fallback for users without a personal one). The *sender's* signature is appended at send time — see `backend/app/services/signatures.py`.
- **Monitoring**: the **/analytics** page (all staff) tracks AI token usage against the daily budget, email volume, draft edit/rejection rates, and escalation trends over 7/30/90-day ranges. First place to look when usage questions come up.
- **Triage rules**: Settings → Triage Rules — per-category T1 eligibility + confidence thresholds (complaint/urgent/promotional are locked server-side).
- **Knowledge base**: Knowledge page — the content the AI drafts from. Keep it current; it matters more than prompt tweaks.
- **Users**: Settings (admin) — staff vs admin roles.
- **Audit/Export**: Audit Log page (admin); `GET /api/v1/emails/export` streams JSON-lines for compliance (rate-limited).
- **Backups**: Railway handled this implicitly. On self-hosting, schedule `pg_dump` (daily, retained ≥30 days). The database is the entire system state — email bodies, drafts, KB, audit log. Attachment binaries are NOT in the

DB (streamed on demand from the mailbox), so DB backups stay small.

6.5 RunPod (dormant, optional)

The system can run drafting on a self-hosted GPU pod (RunPod) instead of Anthropic — built before the 2026-05-14 decision to standardize on Anthropic for email. The orchestration (auto-start, idle-stop, daily cost cap, Claude fallback) is fully functional but **disabled in production** (`RUNPOD_POD_ID` empty). The dev pod lives on a shared RunPod account already held on the client side (~$2.99/hr H100 when running). Ignore unless the firm revisits self-hosting; everything is documented in `.env.example` .

7. Known issues & sharp edges

#	Issue	Impact / workaround
1	`scripts/seed_demo.py` **writes to whatever** `DATABASE_URL` **points at.**	Running it with a production `.env` inserts demo threads into prod. Only run against a dev database.
2	**IMAP provider lacks** `move_message` (`NotImplementedError`).	Spam/Delete mailbox mirroring works on MS Graph only. Only matters if `EMAIL_PROVIDER` ever flips to `imap`.
3	**Attachment downloads resolve from the live mailbox.**	If a message is hard-deleted from Outlook/Exchange, its attachments 404 in the app (metadata stays, binary is gone). By design — no binary storage.
4	`NEXT_PUBLIC_API_URL` **is build-time.**	Changing the backend URL requires rebuilding the frontend image, not just restarting it.
5	**Deploy drift precedent.**	Verify each deploy actually went live (§4).
6	**Mobile layout is not yet refined.**	Jane accesses via phone browser; it works but isn't optimized. On the roadmap (§8).
7	**Pre-018 drafts have the old global signature baked into their body.**	Handled automatically: the send path strips the known legacy block and appends the sender's signature instead (exact-suffix match only). Self-heals as old drafts drain.
8	**Single-worker constraint** (§6.1).	Scaling out requires extracting the poller first.

8. Open roadmap (not blocking handoff)

1. **General-mailbox persona reword.** Per-user signatures (shipped June 2026) deliberately anticipate the mailbox moving from Jane's personal address to a general firm address (`office@` / `info@`) — staff-signed mail from a shared mailbox is the standard pattern. **One follow-up belongs with that mailbox switch:** the AI drafting persona is currently "write as Jane personally" (`draft_generator.py` system prompt); a general mailbox needs it reworded to a firm-office persona. Prompt-level change only; the mailbox itself is pure config (`MSGRAPH_MAILBOX`).
2. **Mobile view refinement** — committed verbally in the 2026-05-27 client meeting; not yet built.
3. Direction discussed with the firm: evolve toward Jane's primary mail client (folders parity, richer search). Discussion stage only.
4. "What's New" release-notes surface — designed, not built.
5. In-app practice mode for the Compose flow on the tutorials page.

9. Deploy helpers & troubleshooting

Two scripts at the repo root support deployments (any host, not just Railway):

- **`scripts/preflight_check.py`** — run locally against your populated `.env` *before* the first deploy. Validates `APP_SECRET_KEY` strength, real (non-placeholder) AI key, provider credentials matching the chosen `EMAIL_PROVIDER`, no localhost in `CORS_ORIGINS`, non-example admin password. Exits non-zero on failure.
- **`scripts/post_deploy_verify.py --base-url <backend-url> --database-url "$DATABASE_URL"`** — run *after* every deploy. Checks `/health`, confirms `auto_send_enabled='false'`, and confirms zero categories have `t1_eligible=true`. If either safety gate is in an unexpected state, don't hand out the URL until you've confirmed the change was intentional.

Common failures:

Symptom	Usual cause
Backend container restart-loops	`APP_SECRET_KEY` still the dev default (validator hard-fails in production); or `alembic upgrade head` can't reach the DB (`DATABASE_URL`); or `ADMIN_PASSWORD` empty (seed script exits 1)
Drafts not generating	Daily token budget exhausted (check the Analytics page — resets midnight UTC); or `DRAFT_AUTO_GENERATE=false`. Note `SHADOW_MODE` does NOT stop draft generation — it only gates auto-send.
No emails arriving	Graph: admin consent not granted on the application permissions, or the client secret expired. IMAP: using the account password instead of an app password. Check Settings → Integrations.
Frontend "Failed to load"	`NEXT_PUBLIC_API_URL` set as a runtime variable instead of a Docker **build** variable, or `CORS_ORIGINS` missing the frontend URL
Email poller suspected in an incident	Set `EMAIL_POLL_INTERVAL_SECONDS=99999` (effectively pauses polling), redeploy, diagnose without traffic, restore

10. Quick reference

- **Run tests:** `cd backend && venv/Scripts/python -m pytest tests/ -q` — 384 tests, in-memory SQLite, all providers mocked, zero real sends. Safe anywhere, run before every deploy.
- **DB schema / API reference:** README.md §Database Schema + live OpenAPI at `<backend>/docs`.
- **App usage & integrations setup:** USER_GUIDE.pdf / `docs/USER_GUIDE.md`.
- **Local dev:** backend on port **8001** (`uvicorn app.main:app --port 8001`), frontend `npm run dev` on 3000.
- **Migration level today:** `018` (`per-user signatures`).
- **Contacts during transition:** RJ Tohay — rj@primelive.ai (current development); Nate Geraldez — nate@primelive.ai (original prototype, stages 1–2).

Generated 2026-06-04 (rev. 4). The README and `.env.example` in the repository are kept current and take precedence over this snapshot if they ever disagree.